

League Stats Tracker

Group Challenge — Suggested Time: 45–60 minutes

Group Size: 2–4 students

Problem Statement

You are given an array of **player objects** from a small basketball league. Each object has the following shape:

```
{
  name:      string    // player's name
  team:      string    // e.g. "Hawks", "Wolves", "Sharks"
  position:  string    // "Guard" | "Forward" | "Center"
  points:    number[]  // points scored in each game this season
  minutesPlayed: number // average minutes played per game (0–48)
}
```

For this challenge, you will work with the following dataset:

```
const players = [
  { name: "Maya", team: "Hawks", position: "Guard", points: [24, 30, 30],
    minutesPlayed: 34 },
  { name: "Owen", team: "Hawks", position: "Forward", points: [16, 18, 20],
    minutesPlayed: 22 },
  { name: "Diego", team: "Hawks", position: "Center", points: [10, 12, 11],
    minutesPlayed: 15 },
  { name: "Priya", team: "Hawks", position: "Guard", points: [20, 22, 21],
    minutesPlayed: 28 },
  { name: "Noah", team: "Wolves", position: "Forward", points: [30, 33, 33],
    minutesPlayed: 36 },
  { name: "Zara", team: "Wolves", position: "Guard", points: [14, 16, 15],
    minutesPlayed: 20 },
  { name: "Liam", team: "Wolves", position: "Center", points: [8, 10, 9],
    minutesPlayed: 12 },
  { name: "Sofia", team: "Wolves", position: "Forward", points: [20, 22, 24],
    minutesPlayed: 26 },
  { name: "Ethan", team: "Sharks", position: "Center", points: [18, 21, 21],
    minutesPlayed: 24 },
  { name: "Ivy", team: "Sharks", position: "Guard", points: [28, 30, 29],
    minutesPlayed: 33 },
  { name: "Ruby", team: "Sharks", position: "Forward", points: [6, 8, 7],
    minutesPlayed: 10 },
  { name: "Kai", team: "Sharks", position: "Center", points: [20, 22, 21],
    minutesPlayed: 27 },
];
```

Open `solution.js` and implement each function below. Open `index.html` in your browser to run the tests and see your score.

Scoring

Each test case is worth **1 point**.

Section	Functions	Test Cases	Points
Core	5	24	24
Stretch Goals	3	16	16
Total	8	40	40

`solution.js` also includes one **worked example** (`getPlayerNames`) that's already finished for you — it's not scored, but read it first if you're not sure where to start.

Example (0 points — already done for you)

`getPlayerNames(players)`

Returns an array of every player's **name**, in the same order as the input array.

This one is already implemented in `solution.js`. It's here to show the basic pattern you'll reuse for the rest of the challenge: take the `players` array, use an array method to pull out exactly what you need, and return a new array.

Example:

```
getPlayerNames(players)
// => ["Maya", "Owen", "Diego", "Priya", "Noah", "Zara", "Liam", "Sofia",
      "Ethan", "Ivy", "Ruby", "Kai"]
```

Core Functions (24 points)

`getPlayerAverage(player)` — 5 points

Returns the **average points per game** for a single player. Return `0` if the `points` array is empty.

💡 **Tip:** You'll find this function useful when writing many of the others below. Try destructuring `points` straight out of the parameter: `function getPlayerAverage({ points }) { ... }`.

Example:

```
getPlayerAverage({ name: "Maya", team: "Hawks", position: "Guard", points: [24,
30, 30], minutesPlayed: 34 })
// => 28
```

`getTeamAverage(players, team)` — 5 points

Returns the **average points per game across all players on a given team**. Each player contributes their own average (from `getPlayerAverage`) as a single data point. Return `0` if no players match the team.

Example:

```
getTeamAverage(players, "Hawks")  
// => 19.5    (Maya 28, Owen 18, Diego 11, Priya 21 → sum 78 / 4)
```

`getTopScorer(players)` — 4 points

Returns the **player object** with the highest average points per game. Return `null` if the array is empty. If multiple players share the highest average, return the **first one found**.

Example:

```
getTopScorer(players)  
// => { name: "Noah", ... }    (avg 32, the highest)
```

`getPositionGroup(players, position)` — 5 points

Returns a **new array** containing only players whose `position` matches the given value. The match is **case-sensitive**. Return an empty array if no players match. Do not modify the original array.

Example:

```
getPositionGroup(players, "Guard")  
// => [ Maya, Priya, Zara, Ivy ]
```

`getPlayerSummary(player)` — 5 points

Returns a **formatted string** describing the player, using their name, position, team, and average points per game rounded to **one decimal place**.

Format: "<name> (<position>, <team>) – <avg> ppg"

Example:

```
getPlayerSummary({ name: "Maya", team: "Hawks", position: "Guard", points: [24, 30, 30], minutesPlayed: 34 })  
// => "Maya (Guard, Hawks) – 28.0 ppg"
```

Stretch Goals (16 points)

These functions are optional but encouraged. Each one builds on the core concepts above.

tagAllStars(players, threshold) — 6 points

Returns a **new array** where each player object has an added `allStar` property (`true` / `false`) based on whether their average points per game is **greater than or equal to** `threshold` . Do **not** modify the original array or its objects.

Example:

```
tagAllStars(players, 20)  
// => [  
//   { name: "Maya", ..., allStar: true }, // avg 28  
//   { name: "Owen", ..., allStar: false }, // avg 18  
//   { name: "Diego", ..., allStar: false }, // avg 11  
//   ...  
// ]
```

getBenchWarmers(players, minAvg, minMinutes) — 5 points

Returns a **new array** of players who are considered "bench risks". A player is a bench risk if **either** of the following is true:

- Their average points per game is **below** `minAvg`
- Their `minutesPlayed` is **below** `minMinutes`

Do not modify the original array.

Example:

```
getBenchWarmers(players, 15, 20)
// => [ Diego, Liam, Ruby ]
// Diego: avg 11 < 15 (also minutesPlayed 15 < 20)
// Liam: avg 9 < 15 (also minutesPlayed 12 < 20)
// Ruby: avg 7 < 15 (also minutesPlayed 10 < 20)
```

getTeamStandings(players) — 5 points

Returns a **new array** with one entry per team, sorted by team average points per game in **descending order**, with a `rank` property added (1 = highest). Each entry should look like `{ team, average, rank }`.

Teams with equal averages **share the same rank** — the next rank is skipped (e.g. two teams at rank 1 means the next rank is 3).

Do **not** modify the original `players` array or its objects.

 **Tip:** Try grouping players into a plain object keyed by team name, then use `Object.entries()` to turn that object back into an array you can `map()` and `sort()`.

Example:

```
getTeamStandings(players)
// => [
//   { team: "Hawks", average: 19.5, rank: 1 }, // tied
//   { team: "Wolves", average: 19.5, rank: 1 }, // tied → next rank is 3
//   { team: "Sharks", average: 19.25, rank: 3 },
// ]
```

Tips

- Not sure where to start? Read `getPlayerNames` at the top of `solution.js` first — it's already solved and shows the pattern you'll use everywhere else.
- Stuck, or not sure why a function isn't returning what you expect? `console.log()` your inputs and your outputs. Look at the `console.log(players)` inside

`getPlayerNames` — do the same thing inside your own functions (log the parameters at the top, log the value right before you `return` it) to see exactly what you're working with.

- Start with `getPlayerAverage` — nearly every other function depends on it.
- Read each function's edge cases carefully (empty arrays, exact boundary values, ties).
- "Do not modify the original array" means you should return a **new** array (and new objects, where noted), not mutate the input. The spread operator (`...`) is your friend here.
- Destructuring (`const { name, team } = player;` or destructuring right in a function's parameter list) can make your code much shorter and easier to read.
- Use the browser test runner (`index.html`) to verify your work as you go — don't wait until the end.
- For stretch goals, think about what existing functions you can reuse.
- Divide the work across your team: one person can start on the core functions while others sketch out the stretch goals, then combine and test together.